Ejercicio de Aplicación 🤓

L5: Técnicas de reducción dimensional

Es hora de que pongas en práctica todo lo aprendido. 🤓

Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que a continuación encontrarás diferentes **desafíos** que podrás resolver de forma independiente y a tu ritmo.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😉

¡Manos a la obra!

1. **Desafío 🎯**

   Descargar un ***dataset con al menos 10 variables numéricas*** *(puede ser del UCI Machine Learning Repository o Kaggle) y realizar los siguientes pasos:*

   - *Hacer un análisis exploratorio inicial del dataset.*
   - *Aplicar* ***PCA*** *para reducir las dimensiones a 2 componentes y visualizar los resultados.*
   - *Aplicar* ***t-SNE*** *con parámetros adecuados (por ejemplo, perplexity=30) y visualizar la proyección 2D.*
   - *Comparar visualmente y con una reflexión escrita los resultados obtenidos por ambas técnicas.*

2. **¿Dónde se lleva a cabo? 👩‍💻**

   Visual Studio Code

3. **Tiempo de dedicación ⏳**

   2 Horas.

4. **Recursos🛠️**

   https://archive.ics.uci.edu/dataset/109/wine

5. Plus ➕

- *Prueba aplicar escalado previo (StandardScaler) antes de PCA y t-SNE para mejorar los resultados.*
- *Modifica parámetros de t-SNE como perplexity, learning_rate y n_iter para observar su efecto.*

6. ⚠️Condición

Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Ejercicio de Aplicación 🤓

L5: Técnicas de reducción dimensional

Resolución del desafío 👇

```python
# Paso 1: Importar librerías
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.datasets import load_wine

# Paso 2: Cargar datos
data = load_wine()
X = pd.DataFrame(data.data, columns=data.feature_names)

# Paso 3: Escalar los datos
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Paso 4: Aplicar PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```python
# Visualizar PCA
plt.figure(figsize=(8,5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=data.target, cmap='viridis')
plt.title("Proyección PCA (2D)")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.colorbar(label='Clase')
plt.grid(True)
plt.show()

# Paso 5: Aplicar t-SNE
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
X_tsne = tsne.fit_transform(X_scaled)

# Visualizar t-SNE
plt.figure(figsize=(8,5))
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=data.target, cmap='coolwarm')
plt.title("Proyección t-SNE (2D)")
plt.xlabel("Dim 1")
plt.ylabel("Dim 2")
plt.colorbar(label='Clase')
plt.grid(True)
plt.show()
```